Exercise Sheet 4

No Fear of Numbers: Introduction to Quantitative Data Analysis in R

Dr. Susanne de Vogel, Dr. Maryam movahedifar, Data Science Center, University of Bremen

11 March 2026

Multivariate Analysis: Regression Models

So far, we have looked at our data **one variable at a time** (univariate) and in **pairs** (bivariate).
In this sheet, we take the next step to **multivariate analysis**: we use **regression models** to study how several predictors together are related to an outcome.

In the context of our research story, our main outcomes of interest are still:

- overall life satisfaction (`bpsy01`), and
- having children (yes/no, `blcd06`).

Now we want to answer questions like:

- How is life satisfaction related to **several factors at once** (e.g. work–life compatibility, emotional support, burden to perish, self efficacy, income)?
- Does a predictor still matter for life satisfaction **after controlling for** other variables?
- How do different predictors relate to the **probability of having children**?

In this exercise, you will learn to:

- fit and interpret a **multiple linear regression** with `bpsy01` as the dependent variable,
- fit and interpret a **logistic regression** with `blcd06` (children yes/no) as the dependent variable,
- report and interpret **regression coefficients**, **confidence intervals**, and (for logistic regression) **odds ratios**,
- summarize the main results in simple sentences (e.g. "Holding other variables constant, higher burden is associated with lower life satisfaction").

0. Set-up: Load packages and dataset, load data frame

1. Install `tidyverse`, `haven`, `tidymodels` if not already installed and load these packages. Define the *exercise* folder as your working directory with the function `setwd("path")` and load dataset "04_qa_multi_data.sav" as a data frame called `mydata4`.

Code it (or faster: copy and paste the code) like this:

```r
# Install tidyverse
if (!requireNamespace("tidyverse", quietly = TRUE))
  install.packages("tidyverse")

# Install tidymodels
if (!requireNamespace("tidymodels", quietly = TRUE))
  install.packages("tidymodels")

# Install haven
if (!requireNamespace("haven", quietly = TRUE))
  install.packages("haven")


## load packages
library(tidyverse)
library(haven)
library(tidymodels)

# set working directory
setwd(
  "C:/Users/Susanne/Nextcloud/share_DSC/003_Trainings/001_Workshops/2026/2026-03-11_SdV_MM_Quantitative_Data_Anal
  )

# Load the Nacaps data set and name this data frame "mydata4"
mydata4 <- read_sav("04_qa_multi_data.sav")
```

0 Prep: Converting all categorical to factor variables

In preparation of the multiple linear regression, make sure the categorical variables are factors (`adbi15, adem01, adem03, apar10, bdbi01, blcd01, blcd06`) .

The code should look like this:

```r
mydata4 <- mydata4 %>%
  mutate(
    across(
      c(adbi15, adem01, adem03, apar10, bdbi01, blcd01, blcd06),   # all cat. variables
      ~ haven::as_factor(.x)       # labels -> factor levels
    )
  )

glimpse(mydata4)
```

Exercise 1: Multiple linear regression: life satisfaction

We want to model **overall life satisfaction** (`bpsy01`) as a function of PhD characteristics and work conditions, attitudes and well-being, and control for demographics. We use **multiple linear regression** with `lm()` and **tidy the output** with *tidy*.

1.1. Create a cleaned data frame `mydata_lm_ex1` that only contains rows with non-missing values for the dependent variable and all predictors in this model.

The code should look like this:

```
mydata_lm_ex1 <- mydata4 %>%
  filter(
    !is.na(bpsy01),
    !is.na(bdbi01),
    !is.na(adbi15),
    !is.na(bdcd17),
    !is.na(bdwr12),
    !is.na(bemp81),
    !is.na(blcd12),
    !is.na(apar15b),
    !is.na(bpsy05),
    !is.na(adem01),
    !is.na(adem02),
    !is.na(apar10),
    !is.na(adem03),
    !is.na(blcd01),
    !is.na(blcd06)
  )
```

The new data frame `mydata_lm_ex1` should now appear in your environment pane.

1.2. Fit a multiple linear regression with `bpsy01` as the dependent variable and all other variables as predictors/controls.

The code should look like this:

```
lm_phd <- lm( # save results in data frame lm_phd
  bpsy01 ~ bdbi01 + adbi15 + bdcd17 + bdwr12 + bemp81 + #bpsy01 as dependent var, predictors: PhD
↪  and work conditions
            blcd12 + apar15b + bpsy05 +                         # predictors: attitudes and
↪  well-being
            adem01 + adem02 + apar10 + adem03 + blcd01 + blcd06, # controls: demographics
  data = mydata_lm_ex1 # use data frame mydata_lm_phd
)
```

1.3. Create a tidy table of coefficients and check the results.

Do it like this:

```
# use tidy data frame lm_phd, save results in data frame lm_phd_results and display 95% confidence
↪  intervals
lm_phd_results <- tidy(lm_phd, conf.int = TRUE) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3)) # optional: round output to three decimals
  )

print(lm_phd_results, n = Inf)  # show all rows
```

```
## # A tibble: 26 x 7
##    term                     estimate std.error statistic p.value conf.low conf.high
##    <chr>                       <dbl>     <dbl>     <dbl>   <dbl>    <dbl>     <dbl>
##  1 (Intercept)                 3.01      0.479     6.27    0         2.07      3.95
##  2 bdbi01I have complet~      -0.168     0.12     -1.41    0.16     -0.403     0.066
##  3 bdbi01I have interru~       0.132     0.238     0.556   0.578    -0.335     0.599
##  4 bdbi01I have quit my~      -0.423     0.414    -1.02    0.307    -1.24      0.389
##  5 adbi15sports               -0.175     0.398    -0.44    0.66     -0.955     0.605
##  6 adbi15law, economics~      -0.096     0.131    -0.734   0.463    -0.354     0.161
```

```
## 7 adbi15mathematics, n~   -0.03     0.127   -0.237  0.812  -0.28     0.219
##  8 adbi15human medicine~    0.269    0.151    1.79   0.074  -0.026    0.564
##  9 adbi15agricultural, ~   -0.043    0.255   -0.166  0.868  -0.543    0.458
## 10 adbi15engineering sc~   -0.344    0.145   -2.37   0.018  -0.628   -0.059
## 11 adbi15art, art theory    0.283    0.278    1.02   0.309  -0.262    0.828
## 12 bdcd17                   0.188    0.032    5.93   0       0.126    0.25
## 13 bdwr12                  -0.102    0.048   -2.12   0.034  -0.196   -0.008
## 14 bemp81                   0        0        4.82   0       0        0
## 15 blcd12                   0.332    0.017   19.6    0       0.298    0.365
## 16 apar15b                  0.086    0.053    1.63   0.103  -0.017    0.19
## 17 bpsy05                   0.645    0.06    10.7    0       0.527    0.763
## 18 adem01male              -0.08     0.081   -0.989  0.323  -0.24     0.079
## 19 adem01other              0.076    0.172    0.443  0.658  -0.262    0.414
## 20 adem02                  -0.036    0.008   -4.76   0      -0.051   -0.021
## 21 apar10Bachelor / Mas~    0.174    0.112    1.55   0.121  -0.046    0.394
## 22 apar10Other vocation~    0.116    0.114    1.02   0.309  -0.107    0.339
## 23 apar10Unclassifiable     0.126    0.26     0.484  0.629  -0.385    0.636
## 24 adem03in another cou~   -0.093    0.107   -0.866  0.387  -0.304    0.118
## 25 blcd01no                -0.527    0.091   -5.78   0      -0.705   -0.348
## 26 blcd06no                -0.647    0.111   -5.84   0      -0.864   -0.43
```

You get the following information:

- *term*: variable name (intercept and predictors)
- *estimate*: regression coefficient (change in bpsy01 if the predictor increases by 1, holding - other variables constant; for factors: difference to the reference category, this is always the first)
- *std.error, statistic, p.value* = test statistics
- *conf.low, conf.high* = lower and upper bound of the 95% CI

How do you interpret the results?

1.4. Use the `glance()` function to get an overview of the model fit.

```
lm_phd_fit <- glance(lm_phd)
lm_phd_fit
```

Important values:

- *r.squared*: proportion of variance in life satisfaction explained by the model
- *adj.r.squared*: adjusted R^2 (corrected for number of predictors)
- *p.value* (for the overall F-test): tests if the model explains more variance than a model with no predictors*

How would you interpret the results?

Exercise 2: Multiple logistic regression: probability of having children

We want to predict the probability of **having children (yes/no)** (`blcd06`) as a function of PhD and work conditions, attitudes and well-being and demographics.

We estimate the model with the `glm()` function and use `tidy()` to get odds ratios.

2.1. Create a cleaned data frame `mydata_lm_ex2` that only contains rows with non-missing values for the dependent variable and all predictors in this model.

2.2. Fit the full logistic regression model on having children (yes/no) `blcd06`.

To model the log-odds of `blcd06` for "yes" with reference category "no", we have to relevel our dependent variable with `relevel()`.

The code goes like this:

```
glm_full <- glm(
  relevel(blcd06, ref = "no") ~                   # change reference category
    bdbi01 + adbi15 + bdcd17 + bdwr12 + bemp81 + # predictors
    blcd12 + apar15b + bpsy05 + bpsy01 +
    adem01 + adem02 + apar10 + adem03 + blcd01,
  data   = mydata_glm_ex2,                         # take dataset without missings
  family = binomial(link = "logit")                # logistic regression
)
```

*2.3. Tidy the table and display odds ratios**.

The code should look like this:

```
glm_ex2_results <- tidy(
  glm_full,
  exponentiate = TRUE, # exp(coef) = odds ratios instead of log-odds
  conf.int     = TRUE  # 95% confidence intervals
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) # round numeric values to three decimals for
   ↪  readability


print(glm_ex2_results, n = Inf)  # show all rows
```

In the output table, estimate now shows **odds ratios** (because we used exponentiate = TRUE).

Odds ratio > 1: the predictor is associated with higher odds of having children (blcd06 = "yes").

Odds ratio < 1: the predictor is associated with lower odds of having children.

The 95% confidence interval shows the uncertainty around each odds ratio; if it includes 1, the effect is not statistically significant at the 5% level.

How do you interpret the results?

4. Finally, calculate the overall model fit including McFadden R^2.

The code should look like this:

```
glm_ex2_fit <- glance(glm_full)

glm_ex2_fit %>%
  mutate(
    mcfadden_r2 = 1 - deviance / null.deviance
  )
```

How do you interpret the model fit?